

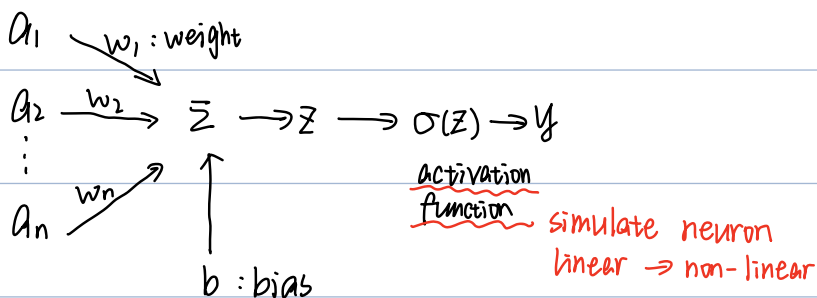
Lecture 1

P1. Introduction of Deep Learning

Step 1: Neural network 前向传播得结果, 算损失

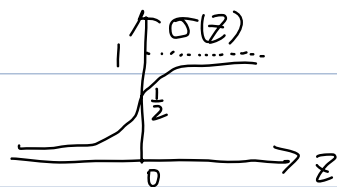
Neuron

$$z = a_1 w_1 + \dots + a_n w_n + b$$



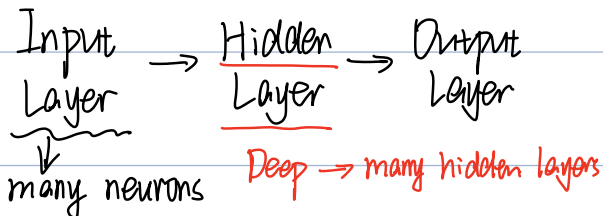
Activation function: sigmoid independent probability distribution

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$$\sigma'(z) = \text{sigmoid}(z) [1 - \text{sigmoid}(z)]$$

Fully connected Network



Activation function: Softmax mutually exclusive probability distribution

Softmax layer: output value $\rightarrow$ probability

Hot Encoding: biggest probability $\rightarrow$ classification result
离散特征取值有大小意义 无大小意义: One-Hot Encoding e.g. male/female

softmax $\rightarrow$ Normalized exponential function 归一化指数函数.

each component 1. $0 \leq z_i \leq 1$

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

2. $\sum_{j=1}^n z_j = 1$

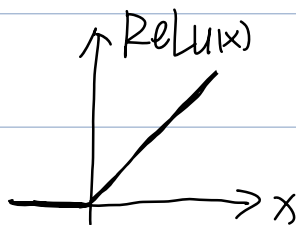
Activation function (others):

ReLU

$$\text{ReLU}(x) = \max(x, 0)$$

sigmoid: gradient vanishing / explosion

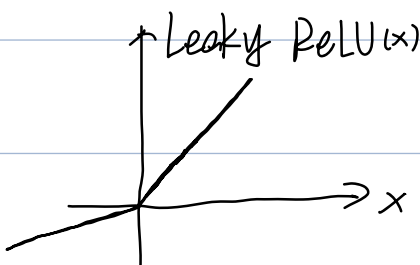
$\downarrow$
ReLU



Leaky ReLU

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

α fixed hyper-parameter



PReLU

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

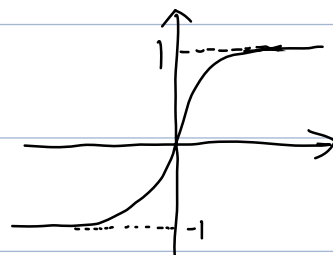
α learnable



tanh

$$\tanh(x) = \frac{1 - e^{(-2x)}}{1 + e^{(-2x)}}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$



Step 2: Cost function

prediction: MSE $\sum_{i=1}^n (y_i - \hat{y}_i)^2$

classification: CE $-\sum_{i=1}^n \hat{y}_i \ln \hat{y}_i$

regularization: L_1 (abs), L_2 (Euclidean distance) $\rightarrow$ reduce overfitting

Step 3: optimization 反向传播更新权重

find network parameters $\rightarrow$ minimize total loss (L)

Gradient descent $\leftarrow$



compute $\frac{\partial L}{\partial w}$ Repeat until $\frac{\partial L}{\partial w}$ approximately small (update little)

P2. Training Deep Neural Network.

误差

- 训练误差: 模型在训练数据上的误差
- 泛化误差: 模型在新数据上的误差

- 过拟合 (overfitting)
- 将模型在训练数据上拟合的比在潜在分布中更接近的现象

- 正则化 (regularization)
- 用于对抗过拟合的技术

- 欠拟合 (underfitting)
- 训练和验证误差之间的泛化误差很小

- 大样本数据
- 训练数据: 训练模型参数
- 验证数据: 一个用来评估模型好坏的数据集
- 不要跟训练数据混在一起 • 选择模型超参数
- 测试数据: 只用一次的数据集

模型 \ 数据	简单	复杂
	低	高
低	正常	欠拟合
高	过拟合	正常

- 小样本数据
- K-fold cross-validation
- 将训练数据分割成 K 块
- For $i=1, \dots, K$ • 使用第 i 块作为验证数据集，其余的作为训练数据集
- 报告 K 个验证集误差的平均

超参数 (hyperparameter)

- 可以调整但不在训练过程中更新的参数
- 学习率: η
- 学习率太大会使模型发散，学习率太小会没有进展。
- 批量大小
- Epoch

- 调参 (hyperparameter tuning)
- 选择超参数的过程。

超参数通常是我们根据训练迭代结果来调整的，而训练迭代结果是在独立的验证数据集 (validation dataset) 上评估得到的

bad result on training data:

1. choosing proper loss (e.g softmax $\rightarrow$ cross entropy)

2. Mini-batch : 小批量使 GPU 并行处理 $\rightarrow$ 损失函数 $\rightarrow$ 参数更新. $\xrightarrow{\text{batch 全跑完一次}}$
epoch

3. new activation function

ReLU $\rightarrow$ Thinner linear network

bad result on testing data

1. Early stopping

2. weight decay

$$\text{ori: } w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

$$\text{wd: } w \leftarrow \underbrace{(1-\lambda)}_{\text{smaller}} w - \eta \frac{\partial L}{\partial w}$$

3. dropout *only on training!*

before updating the parameters, each neuron has ^{hyper-parameter} p% to dropout

